

ICG202 Introduction to Computer Graphics
Assessment 2: Interactive Theory Exploration

Giada Arosio (A00168823)

Transformation Pipeline

The 4K image used for this pipeline (Figure 1) is the official artwork from the Fable Premium Edition by Playground Games (4K Wallpapers, 2026), and it shows the game's Hero and the fantasy world of Albion.

Figure 1

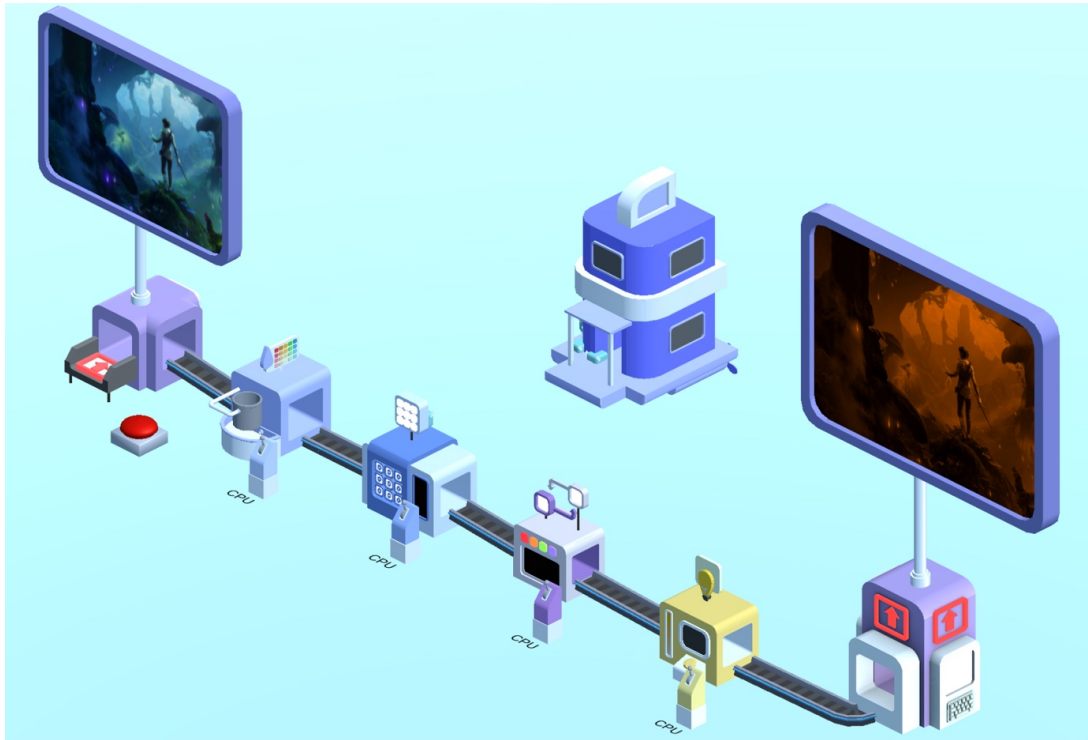
4K Artwork from Fable Premium Edition



The goal was to transform this image into an antique bronze coin using the Unity Graphics Pipeline Simulator. This required building a pipeline (Figure 2) that creates a metallic look with stronger texture, highlights and contrast.

Figure 2

Complete Image Transformation Pipeline in Unity Simulator



The behaviour described for each step below is based on both the modifier's source code (visible via the CODE tab in the simulator) and the visual output observed at each stage.

Step 1 – Colorify

Colour selected: #FF7700

Colorify replaces every pixel's colour with a scaled version of the selected colour, making it proportionally darker or brighter depending on the original intensity of that pixel. As shown in Figure 3, this shifted the original colour palette to different shades of the chosen warm copper-orange tone, creating the base colour for the aged metal effect.

Figure 3

Output After Step 1: Colorify Filter Applied (#FF7700)



Step 2 – Kernel

Matrix: Emboss (Powell, 2015)

Iterations: 1

Kernel recalculates each pixel using information from the surrounding pixels (3×3) and the given matrix. The Emboss kernel creates a "relief" effect by using different weights on opposite corners, making edges lighter on one side and darker on the other while highlighting differences between neighbouring pixels (Powell, 2015). This enhanced the texture of the image, making it look as if it had been engraved into metal (Figure 4).

Figure 4

Output After Step 2: Emboss Kernel Filter Applied



Step 3 – Replace Colour

Original Colour: #FFE044

Replacement Colour: #ECECA7

Tolerance: 80

Replace Colour compares each pixel to the selected original colour. If it falls within the selected tolerance range, it is replaced with the new colour. Otherwise, it is left unchanged. As shown in Figure 5, this was used to increase the brightness of selected areas of the image, making some details of the armour and background more defined to better simulate metallic highlights of the coin.

Figure 5

Output After Step 3: Replace Colour Filter Applied



Step 4 – Brightness

Setting: -5

Brightness adjusts the red, green and blue (RGB) values of every pixel by the selected amount, making the entire image brighter or darker. The -5 setting only slightly darkened the image (Figure 6), but it deepened the shadows just enough to make the highlights from the previous step stand out and the overall effect more realistic.

Figure 6

Final 4K Output After Step 4: Brightness Filter Applied (-5)



NOTE: Since the simulator does not support image cropping, the final result keeps its rectangular shape, making it closer to an ancient commemorative plaque than a coin.

The identical pipeline was executed on both CPU and GPU, and then applied to the HD resolution (Figure 7). Across all combinations, small visual discrepancies were observed between outputs, likely due to hardware-specific floating-point rounding during colour calculations. However, parameters were kept constant to ensure comparability of performance data.

Figure 7

Final HD Output



Notably, an earlier pipeline placed Replace Colour before Kernel, producing red fringing edge artifacts, a "false colour" artefact likely created by Kernel when blending very different colours (Figure 8; Novikov et al., 2024). Reversing the order removed this issue.

Figure 8

Red Fringing Edge Artefact Caused by Premature Replace Colour Processing



Computer Resources

The following tables and graphs report and compare the execution times, in milliseconds, recorded for the CPU and GPU versions of the transformation pipeline. Each configuration was executed five times, on both the HD and 4K images, and the results were averaged to reduce the impact of fluctuations between runs.

HD results:

Table 1

CPU and GPU Execution Times for the HD (1280 × 720) Image Transformation Pipeline Across Five Runs

HD	Colorify		Kernel		Replace Colour		Brightness		Total	
	CPU	GPU	CPU	GPU	CPU	GPU	CPU	GPU	CPU	GPU
1	13.6	8.8	78.9	11.5	15.7	11.5	11.9	11.8	120.1	43.6
2	13.9	9.7	73.3	8.8	15.5	12.3	12.0	11.5	114.7	42.3
3	113.3	10.9	76.7	10.7	105.4	11.8	12.1	11.4	307.5	44.8
4	14.1	11.6	70.2	12.8	20.6	11.8	11.9	15.9	116.8	52.1
5	13.5	11.8	116.1	11.5	20.7	11.6	11.9	11.7	162.2	46.6
Mean	33.68	10.56	83.04	11.06	35.58	11.80	11.96	12.46	164.26	45.88

Figure 9

HD Image: CPU vs GPU Execution Time Across Five Runs

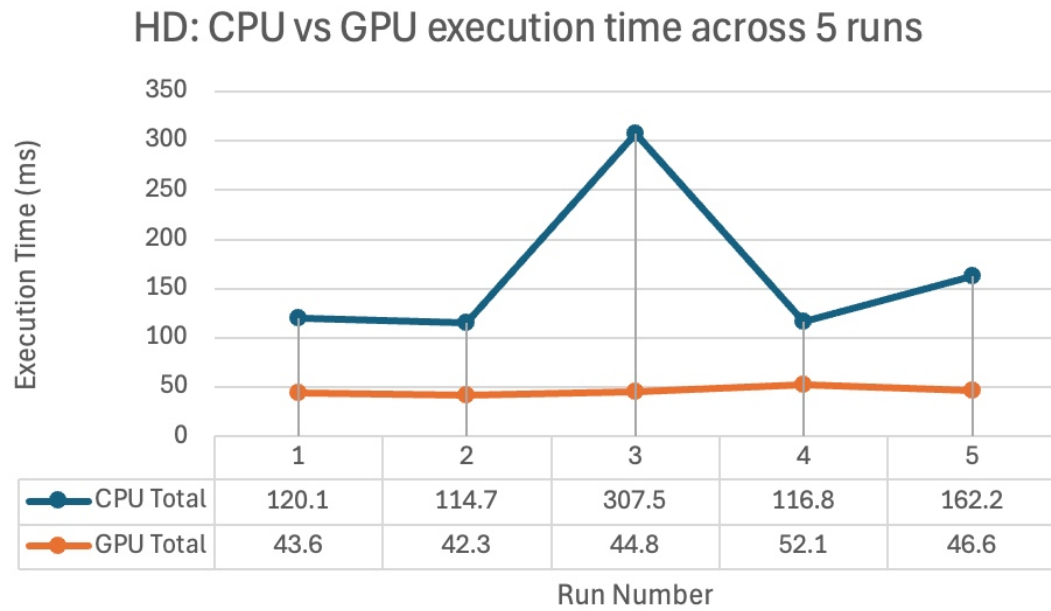
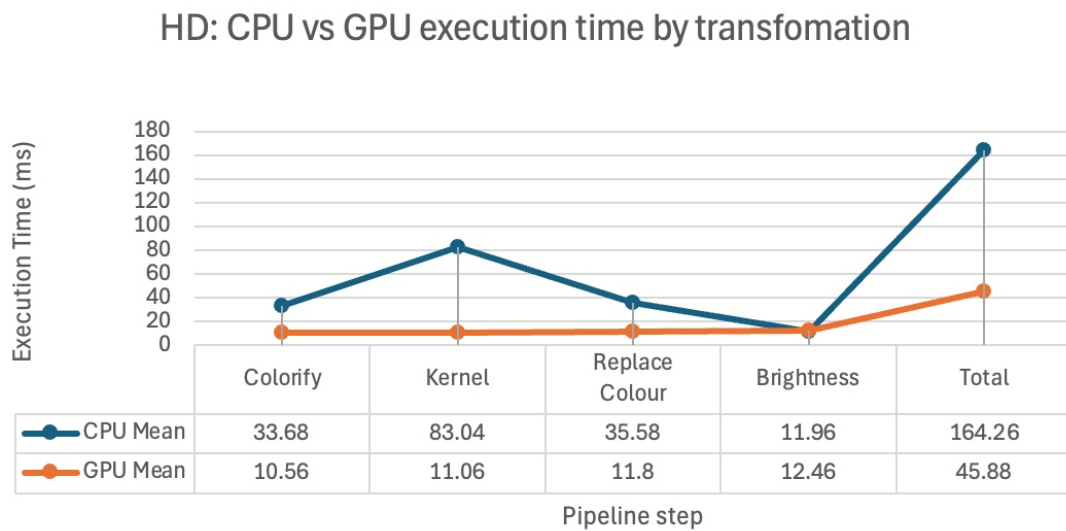


Figure 10

HD Image: Average CPU vs GPU Execution Time by Transformation



4K results:

Table 2

CPU and GPU Execution Times for the 4K (3840 × 2160) Image Transformation Pipeline

Across Five Runs

4K	Colorify		Kernel		Replace Colour		Brightness		Total	
	CPU	GPU	CPU	GPU	CPU	GPU	CPU	GPU	CPU	GPU
1	50.2	118.6	504.4	36.6	61.4	30.9	63.6	37.7	679.6	223.8
2	49.5	48.3	504.0	37.6	60.4	35.4	58.4	43.2	672.3	164.5
3	49.5	36.3	548.6	39.3	61.4	39.3	62.8	25.3	722.3	140.2
4	49.4	20.6	896.5	30.6	57.1	22.9	85.9	31.0	1088.9	105.1
5	412.6	16.5	491.9	28.1	330.2	47.5	59.4	26.6	1294.1	118.7
Mean	122.24	48.06	589.08	34.44	114.10	35.20	66.02	32.76	891.44	150.46

Figure 11

4K Image: CPU vs GPU Execution Time Across Five Runs

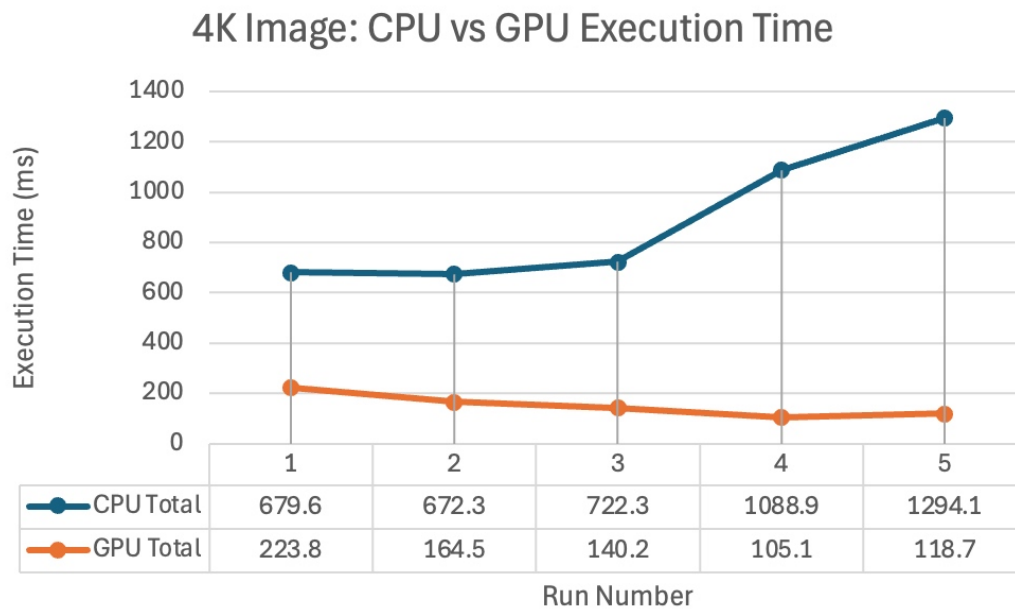
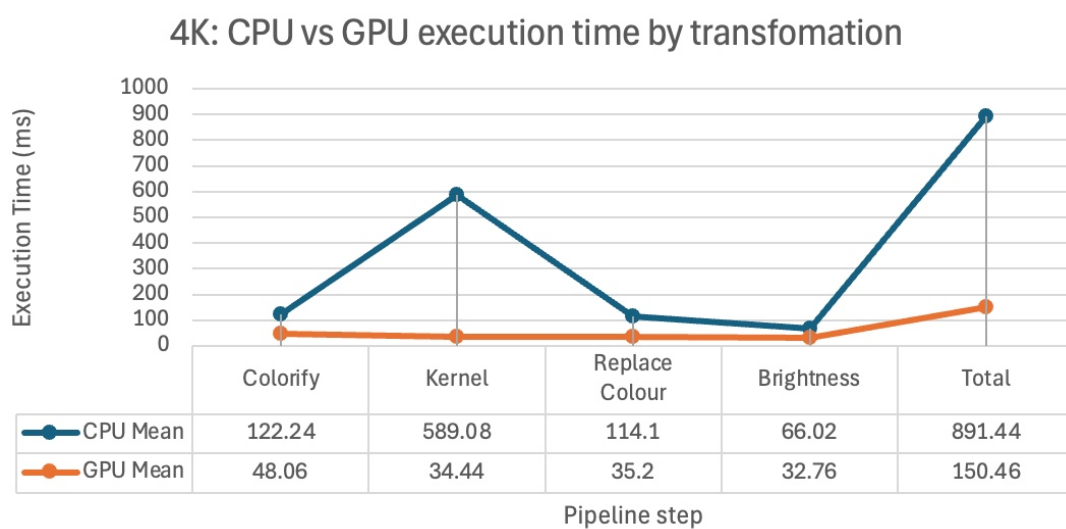


Figure 12

4K Image: Average CPU vs GPU Execution Time by Transformation



Reflection

The results show that on both HD and 4K images, the GPU pipeline was overall faster than the CPU, however, the size of that advantage varied significantly, depending on the filter and the resolution.

The reason comes down to the code, how much work each filter does per pixel, and the hardware architectures. CPUs perform complex, sequential processing using a few powerful cores (Akenine-Möller et al., 2018; Kirk & Hwu, 2016), while GPUs run operations in parallel across thousands of simpler cores (Espinoza, 2024).

Brightness only involves one simple calculation per pixel, so there is little to gain from parallelising it, and the GPU's fixed cost of transferring data and preparing the shader outweighs the time saved by running in parallel (Slaughter et al., 2020). Conversely, Kernel created the largest CPU bottleneck. Recalculating each pixel from its 3×3 neighbourhood (tlPixelIndex, trPixelIndex) is exactly the repetitive work that benefits from GPU parallel execution via fragment shaders (Akenine-Möller et al., 2018). Colorify and Replace Colour sit between these two workload extremes, with the GPU being about 3x faster than the CPU (Tables 1 and 2).

This mechanism explains what happens when resolution increases from HD to 4K. Brightness went from almost no performance benefit (11.96ms vs 12.46ms, Table 1) to nearly twice as fast on GPU at 4K (66.02ms vs 32.76ms, Table 2). The fixed GPU cost stays roughly the same regardless of image size, but 4K has nine times more pixels than HD, so provides enough computation to make the parallel processing worthwhile, even for a simple filter. Kernel shows the same scaling effect: its CPU/GPU gap widens from about 7.5× on HD to roughly 17× on 4K (83.04ms vs 11.06ms, then 589.08ms vs 34.44ms; Figure 10 and 12), as more pixels mean more work for the GPU to distribute.

Comparing scaling directly, see Tables 1 and 2, total CPU execution time increased by $5.4\times$ from HD to 4K (164.26ms to 891.44ms), whereas GPU time increased by only $3.3\times$ (45.88ms to 150.46ms), despite the image containing 9 times more pixels, neither processor scales linearly with this increase. The GPU scales far more efficiently by maximizing core use, while the CPU is forced to process every additional pixel sequentially.

The CPU also demonstrated less consistency across the five runs, with spikes likely indicating external operating system tasks interrupting the single thread, something the GPU's dedicated hardware is not exposed to (Figures 9 and 11).

Based on the results, the GPU is the better choice overall when working with large images or computationally expensive effects. Its performance advantage becomes significantly more pronounced as the workload increases, particularly for transformations like Kernel. However, for simpler operations, the setup overhead of the GPU might not be worth it and can exceed the execution time of a quick CPU loop (Slaughter et al., 2020), which may remain effective without adding GPU complexity.

Performance is not the only factor to consider. CPUs are generally more accessible and cost-effective, while GPUs have higher energy costs and environmental impact (Schneider & Smalley, 2025). Therefore, in future projects, neither processor should be applied automatically. Instead, a hybrid approach can be considered to leverage each architecture for what it does best. Overall, the most appropriate solution depends on both the complexity of the workload and the available resources (Schneider & Smalley, 2025).

Reference

- 4K Wallpapers. (2026). *Fable Premium Edition wallpaper* [Image].
<https://4kwallpapers.com/games/fable-premium-26712.html>
- Akenine-Möller, T., Haines, E., Hoffman, N., Pesce, A., Iwanicki, M., Hillaire, S., Baker, D., Drobot, M., & Nehab, D. (2018). *Real-time rendering* (4th ed.). CRC Press.
<https://www.realtimerendering.com/>
- Espinoza, J. (2024). *3D Graphics: Fragment Shaders*. Medium.
<https://medium.com/@jrespinozah/3d-graphics-fragment-shaders-f50fe4a42da0>
- Kirk, D. B., & Hwu, W. -M. W. (2016). *Programming Massively Parallel Processors: A Hands-on Approach* (3rd ed.). Morgan Kaufmann.
<https://shop.elsevier.com/books/programming-massively-parallel-processors/kirk/978-0-12-811986-0>
- Novikov, M., Hanwell, D., & Lluís Gomez, A. L. (2024). *Image processing method and system* (U.S. Patent No. 12,045,948). U.S. Patent and Trademark Office.
<https://patents.justia.com/patent/12045948>
- Powell, V. (2015). *Image kernels: Explained visually*. Setosa. <https://setosa.io/ev/image-kernels/>
- Schneider, J., & Smalley, I. (2025). *CPU vs. GPU for machine learning*. IBM Think.
<https://www.ibm.com/think/topics/cpu-vs-gpu-machine-learning>
- Slaughter, E., Wu, W., Fu, Y., Brandenburg, L., Garcia, N., Kautz, W., Marx, E., Morris, K. S., Cao, Q., Bosilca, G., Mirchandaney, S., Lee, W., Treichler, S., McCormick, P., & Aiken, A. (2020). *Task Bench: A Parameterized Benchmark for Evaluating Parallel Runtime Performance*. arXiv. <https://arxiv.org/pdf/1908.05790>